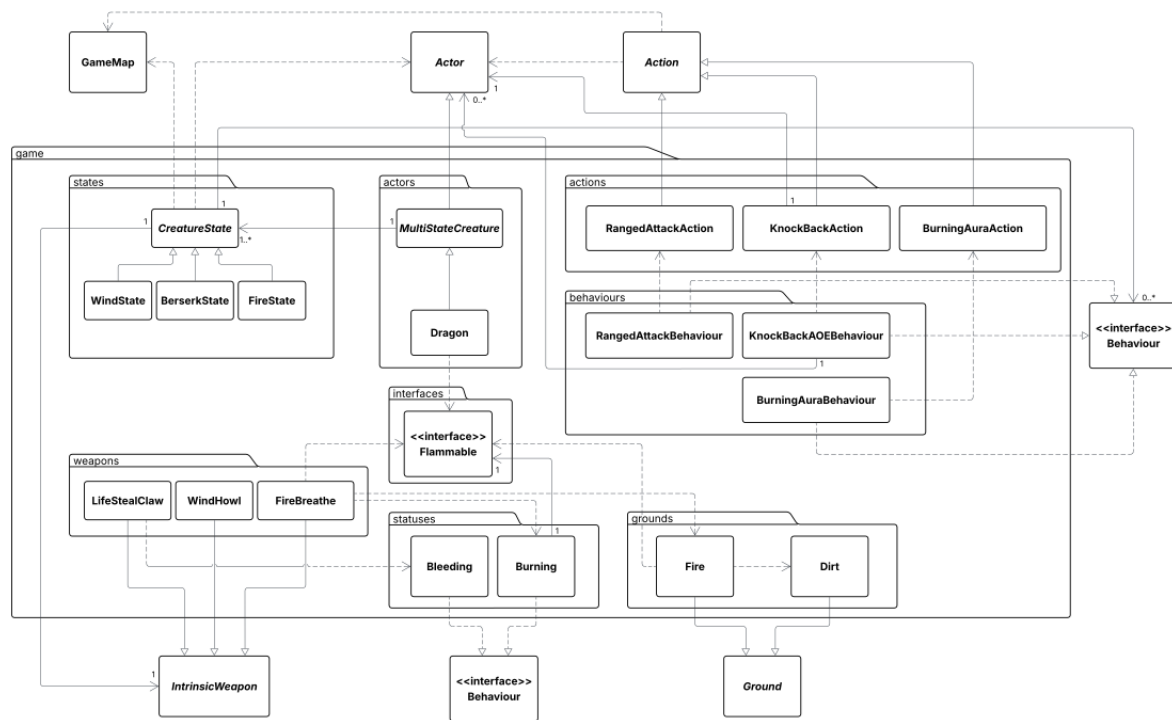
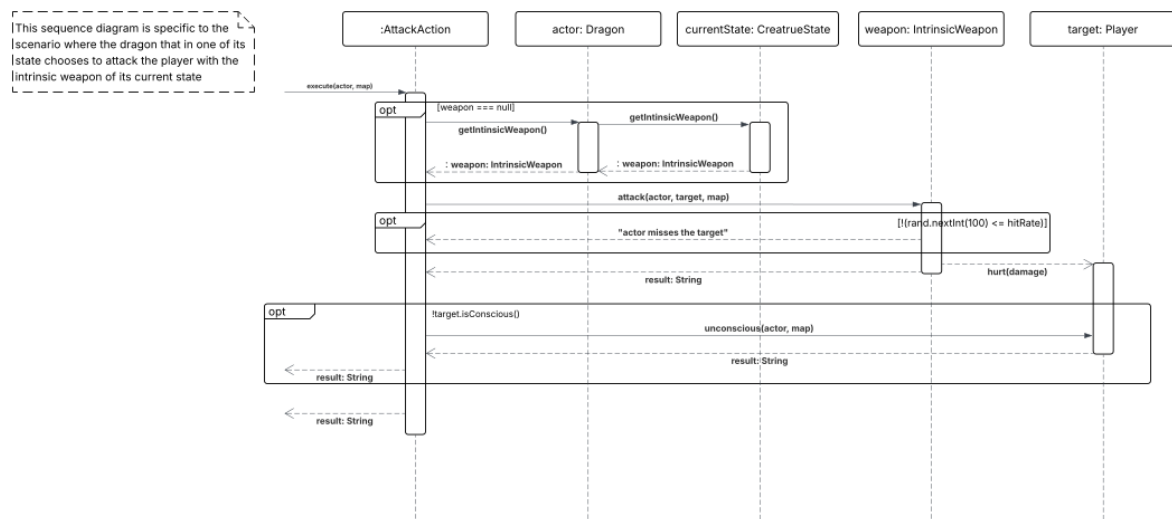


REQ 5 — Design Rationale: Stateful Creatures

UML diagram



Sequence Diagram



Design 1

In this alternative design, CreatureState is defined as an interface, and each specific state (e.g., FireState, WindState, BerserkState) extends it to define its own effects and behaviours. The MultiStateCreature class holds a reference to the current CreatureState, and state changes are handled by switching which interface implementation the creature currently uses. Each creature may implement multiple state interfaces, representing all the forms it can take.

Advantages

- **High Flexibility (OCP, ISP)**
New states can be added as separate interfaces without modifying existing classes. Each interface defines only relevant behaviours, making the design modular and adaptable for future hybrid or shared-state creatures.
- **Loose Coupling (DIP)**
High-level components depend on the CreatureState abstraction rather than concrete subclasses. This allows generic handling of states without tightly binding the creature to specific implementations.
- **Fine-grained Reusability**
States can be selectively combined across different creatures, providing flexible composition of abilities and behaviours.

Disadvantages

- **Difficult State Transition Management**
Since each state exists only as an interface, there is no built-in mechanism for switching between them. The MultiStateCreature must handle transitions manually (e.g., mapping conditions to interfaces), which increases coupling and makes the system harder to maintain or debug.
- **redundant Behaviour and Action Classes**
Because the original AttackAction and AttackBehaviour cannot identify which state is the actor in, so each new state often requires its own AttackAction and AttackBehaviour. Otherwise, we need to use instanceof to check every possible state. This leads to code duplication and reduces scalability as the number of states grows.
- **Limited Polymorphism**
Interfaces define structure but not behaviour, so state-specific actions often rely on conditional checks (e.g., instanceof), violating OCP and reducing the clarity of polymorphic dispatch.

Design 2

Design Overview

This design introduces an abstract class **MultiStateCreature** that generalises all creatures capable of changing between multiple combat forms.

Each creature maintains a CreatureState object representing its current form (e.g., FireState, WindState, BerserkState) and a predefined sequence of possible states.

The creature's **maximum health** is evenly divided among these states.

When its health decreases past each threshold, it transitions automatically to the next state in order.

Each **CreatureState** defines its own intrinsic weapon, passive abilities, and special behaviours. When a state becomes active, enterState() grants its associated abilities and behaviours, while leaveState() removes those of the previous state, maintaining consistent isolation of effects.

Connascence Analysis

- This design demonstrates controlled and well-structured forms of connascence across both static and dynamic dimensions. Most dependencies are intentionally introduced to maintain cohesion between MultiStateCreature, CreatureState, and the engine's Behaviour framework, while avoiding unnecessary coupling.
- **Connascence of Name** arises through consistent method naming such as enterState(), leaveState(), and getIntrinsicWeapon(), enabling polymorphism and uniform state management. All state classes follow the same naming convention, ensuring clarity and predictability in transitions.
- **Connascence of Type** is in the shared use of abstract types such as Behaviour, CreatureState, and IntrinsicWeapon. By depending on abstractions instead of concrete classes, the design adheres to the Dependency Inversion Principle, reducing the impact of future code changes.
- **Connascence of Execution Order** is critical in the transition process. Each change must call leaveState() before enterState() to correctly remove the previous state's abilities and behaviours. This sequencing is explicit and ensures consistent, bug-free state management.

Advantages

SRP (Single Responsibility):

- MultiStateCreature handles lifecycle and transition control.
- CreatureState manages per-state logic, intrinsic weapons, and behaviours.
- Each IntrinsicWeapon encapsulates its own combat effect.
- Behaviour subclasses implement isolated environmental or aura actions. This avoids monolithic creature classes and promotes cohesion.

OCP (Open Closed):

- New states or entire multi-state creatures can be added by subclassing CreatureState or MultiStateCreature. No existing logic needs to change, only extension through new subclasses or behaviours.

DIP (Dependency Inversion):

- All high-level components (e.g., MultiStateCreature) depend on abstractions like CreatureState or Behaviour rather than concrete types. This encourages reuse and plug-and-play modularity.

Behaviour Modularity and Reusability

Each state can compose multiple behaviours with defined priority levels. For instance, a WindState could combine both KnockBackAOEBehaviour and a movement pattern behaviour by just putting them to its stateBehaviours. This composition model makes behaviours highly reusable across different states or creatures.

Clear Encapsulation and Polymorphism

All attack effect is encapsulated within the state's intrinsic weapon. Each state dynamically defines its own attack and behaviour without affecting others, and transitions occur transparently through polymorphism. This keeps the system easy to extend, test, and reason about.

Enhanced Extensibility and Scalability

Weapons like FireBreathe, WindHowl, and LifeStealClaw can be freely reused across multiple states. Behaviours such as BurningAuraBehaviour or KnockBackAOEBehaviour can be attached to any actor, making the system scalable for new creature types or environmental interactions.

Disadvantages

Increased Structural Complexity

The modular approach introduces multiple new abstractions (CreatureState, various behaviours, and weapons). While this improves flexibility, it increases the number of interrelated classes, which may slightly raise the learning curve for new developers.

Priority Management Complexity

Using a TreeMap<Integer, Behaviour> provides control but introduces dependency on priority values. Conflicting priorities may cause subtle behavioural timing issues if not managed consistently.

Conclusion

This design introduces a multi-state creature system, allowing creatures such as the Dragon to dynamically transition between distinct elemental forms — FireState, WindState, and BerserkState — each with its own intrinsic weapon, ability, and autonomous behaviour.

The shared abstract classes MultiStateCreature and CreatureState establish a consistent architecture for managing lifecycle transitions and per-state logic. Each state defines its own IntrinsicWeapon to determine attack effects and may attach multiple Behaviour objects to express passive or environmental actions such as fire auras or wind bursts.

State transitions are predetermined and triggered by health thresholds, with each transition automatically removing the old state's abilities and behaviours before activating those of the next state.

Overall, this design achieves strong modularity and scalability, providing a robust framework for implementing complex, evolving creatures that exhibit both reactive and continuous behaviours throughout gameplay.